

Deployed Observation Surfaces for Anonymous DHTs

Control-plane knobs in IPFS/libp2p delegated routing and OHTTP evolution

Unified archive note v0.19 (2026-03-05; archive v162)

Abstract

Anonymous-DHT papers often focus on protocol mechanisms (delegation, PIR, cover, padding). Real deployments frequently leak through *control-plane* and *service-concentration* surfaces: shared delegated-routing endpoints, cache layers, endpoint autoconfiguration, feature negotiation, and fallback. This note isolates those deployed surfaces for IPFS/libp2p and provides a compact “what-to-pin” checklist, so other papers in this archive can stay mechanism-focused and non-redundant.

Scope. We focus on IPFS/libp2p content/peer routing where “anonymous lookup” is pursued via delegated routing, role separation, and incremental privacy upgrades. We treat these as *observation surfaces* that must be declared (as projections, state declarations, or separation manifests) before any anonymity budget is meaningful.

Imports. Observation-tier naming and exposure normal forms (ENF-ID) are imported from Synthesis 3; threat-window semantics (TW-ID) from Synthesis 4. Change-control equivalence for plan/state drift is imported from Synthesis 18. Primary-path role separation is imported from Synthesis 22–23, and profiler/equalization contracts from Anonymity B. [1, 2, 3, 5, 6, 7]

Exports. Exports a compact deployment-surface taxonomy and a minimal pinning checklist: what belongs in `state_decl_id`, what belongs in a separation manifest, and what must be treated as part of the declared transcript projection $X = \pi(T)$. *Rule of thumb:* long-lived deployment state (router sets, cache layers, AutoConf refresh, downgrade rules) belongs in `state_decl_id`; a specific external service interface (method + URL + request-shaping defaults + time budgets) should be bound per line item via `contact_surface_id` (Synthesis 12).[4]

How other papers should cite this note. This note owns the *enumeration* of concrete deployed control-plane surfaces. It does *not* re-define receipts or state declarations: line-item tuple structure is standardized by Synthesis 12, and digest-bound change control for plan/state ids is standardized by Synthesis 18.[4, 3] Other papers should therefore cite this note only when they need to justify that a particular knob/endpoint/cache is part of the reader set (and hence must be pinned), and otherwise cite Synthesis 12/18 for the mechanics of pinning.

1 Two kinds of deployed leakage

This archive treats “deployed” leakage as falling into two dominant classes.

1.1 Service concentration (shared readers)

A mechanism may offer meaningful query privacy only if the *reader* does not trivially see the initiator. In practice, shared public endpoints and cache-backed services can concentrate observation at a small set of operators. For example, IPFS Docs list a public delegated-routing endpoint (<https://delegated-ipfs.dev/routing/v1>) backed by *someguy*; when used as a default, that single service becomes a reader unless an additional privacy layer is deployed.[12, 14] The same docs also describe `cid.contact` as a public Network Indexer that implements the same delegated-routing API, making it another shared reader surface.[11] The IPNI docs further note that the public endpoint <https://delegated-ipfs.dev/routing/v1> is operated as a caching proxy for both Amino DHT and IPNI queries, forwarding to infrastructure such as `cid.contact`; this is exactly the kind of “under the hood coupling that should be treated as a reader surface unless explicitly anonymized.”[13, 14] For anonymous-DHT accounting, the correct response is not prose about “using delegated routing”; it is to (i) declare which service(s) are in the path and (ii) decide whether the design relies on *separation* (non-collusion) or on a cryptographic/privacy layer that tolerates a single reader.

1.2 Control-plane drift (silent surface changes)

Even with a fixed mechanism, deployments drift. If endpoint selection, feature negotiation, or fallback behavior changes over time, then the observable transcript projection changes. This archive’s rule is: drift changes claim identity. Bind such knobs into a digest-bound plan/state declaration (Synthesis 18) so changes are diffable by id, not discovered by inference. [3]

Concrete drift example (public indexer incident response). In September 2025, `cid.contact` announced it would add a temporary set of delegated-routing endpoints to restore publisher reachability during an IPNI disruption. This is a clean example of why endpoint-set drift must be digest-bound and reviewable: the reader set and the caching path can change quickly for operational reasons, and that change is privacy-relevant.[16]

2 Delegated routing (Routing V1) as a surface

Delegated routing is a vendor-agnostic HTTP API used to offload content routing / peer routing / naming to a service.[10, 17] Deployed privacy questions usually reduce to three concrete surfaces:

1. **Endpoint identity.** Which delegated-routing base URLs are used (including localhost/self-hosted routing backends)? Are they public shared endpoints? Are they pinned or discoverable? IPFS Docs explicitly names a public-good delegated-routing endpoint at <https://delegated-ipfs.dev/routing/v1> (operated by the IPFS Foundation and backed by *someguy*) and notes that `cid.contact` also implements the Delegated Routing V1 API.[11] Routing V1 also supports *path-scoped capability endpoints* such as `/routing/v1/providers`, `/routing/v1/peers`, and `/routing/v1/ipns` (with the no-path form enabling all operations). Kubo documentation shows AutoConf manifests that select different endpoint *paths* per capability and even per node class (“with DHT” vs “without DHT”). Changing a path changes the observable transcript surface and must be treated as semantic drift of the contact surface, not as a cosmetic URL change.[36, 10] Since Kubo v0.40.0 (and still in the v0.40.1 patch line), the Routing V1 HTTP API is exposed by default on the gateway port at <http://127.0.0.1:8080/routing/v1>, allowing light clients (e.g., browsers) to treat a local gateway as a delegated-routing backend.[34, 33] This can be disabled

via `Gateway.ExposeRoutingAPI`. In a public gateway deployment, leaving it enabled changes the reader set (who sees routing queries) and therefore must be pinned as part of `state_decl_id`. [35]

2. **Cache layers.** Are requests served from a shared cache (CDN / edge / reverse proxy) that sees query patterns? If yes, the cache operator becomes an observation point. The IPFS blog describes caching in the public delegated-routing endpoint to speed repeated browser lookups; this also concentrates observation at the cache operator. [15] In addition, the Helia delegated-routing client caches successful (HTTP 200) responses in browser environments (Cache API) for a short TTL, creating a second, often-overlooked cache layer that affects what is observable when. [31]
3. **Feature growth.** The Routing V1 family evolves (streaming responses, client filtering, closest-peers endpoints). [18, 19, 20] Each new endpoint potentially enlarges the declared projection X (message count, response shape, pacing).

Pinning rule. The base URL set and any “auto” resolution rules belong in `state_decl_id`. If the privacy story assumes a relay/service split, encode it as a separation manifest (Synthesis 22) rather than a prose promise. [5]

3 AutoConf and “auto” placeholders

Kubo supports *AutoConf*: fetching a remote JSON manifest (`autoconf.json`) that supplies network defaults (bootstrap peers, delegated routers, DNS resolvers, etc.) and resolves explicit `["auto"]` placeholders at runtime. [22, 25, 23, 21, 27] AutoConf is not only a convenience feature: it is a control-plane dependency that can silently change which delegated-routing endpoints are used (and therefore who reads queries). On IPFS Mainnet, the default manifest is machine-readable and includes delegated endpoints such as `https://delegated-ipfs.dev` and `https://cid.contact`. [21]

Override precedence (routing endpoint selection). For receipt purposes, treat *the effective router set* as the outcome of a small precedence stack:

Precedence (high→low)	What it can change (and why it must be pinned)
Process environment (e.g., <code>IPFS_HTTP_ROUTERS</code>)	Replaces delegated-router configuration and AutoConf expansion; shifts the reader set immediately.
Explicit config (router backends + per-capability paths)	Chooses concrete router implementations (DHT vs HTTP routing) and path-scoped endpoints; changes transcript shape and termination.
AutoConf <code>auto</code> expansion	Resolves <code>["auto"]</code> placeholders via remote manifest; can silently rotate endpoints over time.

The rule for this series is simple: if a knob participates in this stack, it belongs in `state_decl_id`. [3, 4]

Kubo also exposes a hard override: `IPFS_HTTP_ROUTERS` replaces AutoConf and any manually configured delegated routers when set. [24]

Kubo’s environment-variable layer also exposes a request-shaping override: `IPFS_HTTP_ROUTERS_FILTER_PROTOCOLS` overrides the values sent via the `filter-protocols` parameter defined by IPIP-0484 for Delegated Routing V1 requests. This changes which transfer protocols are returned (and therefore which retrieval path is feasible) and must be treated as declared control-plane state. [24, 19, 10]

Other delegated-routing clients may implement IPIP-0484 protocol filtering either as a fixed default or as an explicit knob. If a deployed client hardcodes a filter set, that set is effectively part of the binary and must be pinned by release/commit inside `state_decl_id`. Rainbow’s documented configuration surface (as of 2026-03-05) includes router endpoint selection and time budgets but does not document a protocol-filter knob; deployments that rely on protocol filtering should therefore pin the deployed client version or explicitly declare whether `filter-protocols` is applied. [37, 19]

Rainbow exposes the rest of this routing surface as first-class configuration: `RAINBOW_HTTP_ROUTERS` selects the delegated-routing endpoint set, `RAINBOW_AUTOCONF_URL` selects the manifest, `RAINBOW_AUTOCONF_REFRESH` controls the rotation cadence for `auto` placeholders, `RAINBOW_HTTP_ROUTERS_TIMEOUT` controls the network-level timeout for individual router HTTP requests, and `RAINBOW_ROUTING_TIMEOUT` controls the application-level time budget for a parallel routing operation. These knobs are privacy-relevant because they can change which services are contacted, how long a primary attempt is allowed to run, and therefore whether (and how often) fallback triggers; they should be pinned (or explicitly excluded) in `state_decl_id`. [37]

At the process level, `Routing.Routers` selects the concrete router backends (HTTP delegated routers, DHT, etc.), and `Routing.AcceleratedDHTClient` changes the contact pattern and termination behavior of the DHT client. [36]

Concrete drift example (defaults move to auto). The Rainbow gateway implementation documents that its default delegated-routing setting moved from a fixed endpoint (`https://cid.contact`) to `auto` expansion via AutoConf. This illustrates how a seemingly “minor” config default change can move a deployment from a single named reader to a dynamically selected endpoint set. [26]

Self-hosted gateways as delegated-routing backends. If a deployment uses Kubo gateways as delegated-routing backends (the v0.40.0 default), the configuration knob `Gateway.ExposeRoutingAPI` and the gateway’s access-control posture belong in the declared state surface alongside any AutoConf resolution rules; otherwise the reader set can change without a claim-id change. [34, 35]

From an anonymity perspective, AutoConf is a *control-plane dependency*:

- it can silently change which endpoints you talk to (surface drift),
- it can concentrate observation if defaults point to a small set of public endpoints,
- it can couple privacy posture to availability events (fallback/downgrade).

Pinning rule (AutoConf). If AutoConf influences the routing surface, then (at minimum) the AutoConf URL, the resolution rules for `["auto"]`, the refresh cadence (e.g., Rainbow `RAINBOW_AUTOCONF_REFRESH`), any *override knobs* (e.g., `IPFS_HTTP_ROUTERS`), and the allowed fallback behavior belong in `state_decl_id`. If the deployment claims “primary-path” behavior (e.g., privacy relays enabled) then downgrade/fallback is a first-class leakage event and must be pinned by the schedule/state discipline, not left implicit. [8, 3]

4 HTTP retrieval is a derived routing surface

Delegated-routing lookups do not just reveal *where* queries go; they can also change *how* blocks are retrieved. Kubo exposes `HTTPRetrieval.Enabled`: when enabled, it will act on `/tls/http` provider multiaddrs returned by `Routing.DelegatedRouters` and perform pure HTTPS block retrieval (trustless-gateway style `GET /ipfs/{cid}` with `format=raw` or `Accept: application/vnd.ipld.raw`),

alongside or instead of Bitswap.[38, 39] This upgrades the “destination” surface from “libp2p peers” to “HTTP services” (often with their own caches, CDNs, and logs), and should be pinned as state whenever destination privacy is claimed. Rainbow exposes an analogous destination surface: `RAINBOW_HTTP_RETRIEVAL_ENABLE` gates HTTP retrieval, and allow/deny lists (e.g., `RAINBOW_HTTP_RETRIEVAL_ALLOWLIST`, `RAINBOW_HTTP_RETRIEVAL_DENYLIST`) determine which HTTP endpoints are eligible; timeouts (`RAINBOW_RETRIEVAL_TIMEOUT`) influence termination and downgrade behavior. Treat these as declared state whenever the receipt’s destination model assumes a particular retrieval regime.[37]

For Anonymous-DHT receipts, it is usually enough to declare (i) whether HTTP retrieval is enabled (Kubo `HTTPRetrieval.Enabled`; Rainbow `RAINBOW_HTTP_RETRIEVAL_ENABLE`), (ii) the accepted protocol/filter rules (including any `filter-protocols` override)[19, 24], and (iii) retrieval timeout/abort semantics (Kubo retrieval timeouts; Rainbow `RAINBOW_RETRIEVAL_TIMEOUT`) as part of the threat window.[2, 37]

5 OHTTP upgrades and chunked-message evolution

Reader privacy for delegated routing contemplates Oblivious HTTP relays.[28, 30] Discovery and configuration can be standardized (OHTTP discovery).[29] The OHTTP application layer is also evolving: *chunked* OHTTP message formats enable incremental processing.[32]

Why chunking matters. Chunking introduces new observable features (chunk counts, boundaries, pacing, partial-response timing) that can become profiling handles unless they are explicitly treated as part of the declared projection and budgeted (Synthesis 3; Anonymity B).[1, 7]

6 Minimal checklist (what-to-pin)

This is the “copy/paste” checklist for other papers.

7 Relation to the related-work map

This note is intentionally not a literature taxonomy. For representative academic comparators (delegation, PIR, obfuscation, measurement), cite the related-work map (Synthesis 7).[9]

References

- [1] Anonymous. Trace Interfaces for Anonymous DHTs: Observation Tiers, Committee-Contact Traces, and Exposure Normal Forms. Series synthesis note (Synthesis 3), 2026. (This archive.)
- [2] Anonymous. Threat Windows for Anonymous DHT Lookups: Horizon Semantics, Retry Surfaces, and Declared Templates. Series synthesis note (Synthesis 4), 2026. (This archive.)
- [3] Anonymous. Digest-Bound Replay Plans and State Declarations: Equivalence, Minimality, and Change Control. Series synthesis note (Synthesis 18), 2026. (This archive.)
- [4] Anonymous. Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets. Series synthesis note (Synthesis 12), 2026. (This archive.)
- [5] Anonymous. Primary-Path Separation Manifests: Digest-Bound Non-Collusion Assumptions for Anonymous DHT Claims. Series synthesis note (Synthesis 22), 2026. (This archive.)

Surface	Where it must appear in the published claim interface
Delegated-routing endpoint set (full URLs incl. path scopes)	<code>state.decl_id</code> (digest-bound); drift forces a new claim id.
Router set + overrides (Kubo <code>Routing.DelegatedRouters/AutoConf/IPFS_HTTP_ROUTERS</code> script.)	<code>state.decl_id</code> ; drift changes the reader set and <code>IPFS_HTTP_ROUTERS</code> .
Rainbow <code>RAINBOW_HTTP_ROUTERS</code> + <code>RAINBOW_AUTOCONF_REFRESH</code> + <code>RAINBOW_HTTP_ROUTERS_TIMEOUT</code> + <code>RAINBOW_ROUTING_TIMEOUT</code>)	
AutoConf URL and auto resolution rules	<code>state.decl_id</code> ; include fallback/downgrade semantics.
HTTP retrieval + protocol filters (Kubo <code>HTTPRetrieval.Enabled</code> , <code>filter-protocols</code> , <code>IPFS_HTTP_ROUTERS_FILTER_PROTOCOLS</code> ; Rainbow <code>RAINBOW_HTTP_RETRIEVAL_ENABLE</code> (+ allow/deny lists) and pinned client version for any implicit protocol-filter defaults)	<code>state.decl_id</code> or explicit line item; drift changes the destination reader set.
Cache/CDN/reverse-proxy readers	If relied upon: separation manifest; otherwise treat as a single reader in the threat model and budget accordingly.
Feature negotiation (streaming/filtering/closest-peers endpoints)	Treat as part of the declared projection X (ENF-ID) and under change control.
OHTTP relay split (client/relay/target non-collusion)	Separation manifest (Synthesis 22) and (if probabilistic) $(0, \rho)$ budget (Synthesis 23).
OHTTP chunking mode and its pacing/shape affordances	Declared projection X + pinned plan/state; otherwise it is uncontrolled feature drift.

Table 1: Deployed surfaces and where to pin them in the One-True-Archive interface stack.

- [6] Anonymous. Probabilistic Separation Budgets: $(0, \rho)$ Checkpoints for Non-Collusion Drift. Series synthesis note (Synthesis 23), 2026. (This archive.)
- [7] Anonymous. Anonymous DHT Profiling and the Equalization Contract. Anonymity series Paper B, 2026. (This archive.)
- [8] Anonymous. Schedule-Only Retries for Anonymous DHT Lookups. Operational series Paper A, 2026. (This archive.)
- [9] Anonymous. Related Work Map for Anonymous DHT Lookups. Series synthesis note (Synthesis 7), 2026. (This archive.)
- [10] G. Eggert, M. H. Derkani, H. Dias, and M. Rataj. *Delegated Routing V1 HTTP API*. IPFS Standards, 17 Dec 2025. <https://specs.ipfs.tech/routing/http-routing-v1/>.
- [11] IPFS Docs. *Nodes: public delegated routing endpoints and indexers*. Documentation, 2025–2026. <https://docs.ipfs.tech/concepts/nodes/>.
- [12] IPFS Docs. *Public IPFS Utilities*. Documentation, 2025–2026. <https://docs.ipfs.tech/concepts/public-utilities/>.

- [13] IPFS Docs. *IPNI (InterPlanetary Network Indexer)*. Documentation, 2025–2026. <https://docs.ipfs.tech/concepts/ipni/>.
- [14] IPFS Project. *someguy: Delegated Routing V1 server (Amino DHT + Network Indexer proxy)*. Source repository, 2023–2026. <https://github.com/ipfs/someguy>.
- [15] IPFS Blog. *Faster Peer-to-Peer Retrieval in Browsers With Caching in the Public Delegated Routing Endpoint*. 5 Sep 2025. <https://blog.ipfs.tech/2025-delegated-routing-caching/>.
- [16] IPFS Discuss. *Adding delegated routing endpoints to cid.contact*. 10 Sep 2025. <https://discuss.ipfs.tech/t/adding-delegated-routing-endpoints-to-cid-contact/19727>.
- [17] IPIP-0337 Authors. *IPIP-0337: Delegated Content Routing HTTP API*. IPFS Standards (IPIP), 18 Oct 2022. <https://specs.ipfs.tech/ipips/ipip-0337/>.
- [18] IPIP-0410 Authors. *IPIP-0410: Streaming NDJSON in Routing HTTP API*. IPFS Standards (IPIP), 12 May 2023. <https://specs.ipfs.tech/ipips/ipip-0410/>.
- [19] IPIP-0484 Authors. *IPIP-0484: Opt-in Filtering in Routing V1 HTTP API*. IPFS Standards (IPIP), 29 Oct 2024. <https://specs.ipfs.tech/ipips/ipip-0484/>.
- [20] IPIP-0476 Authors. *IPIP-0476: Delegated Routing DHT Closest Peers API*. IPFS Standards (IPIP), 20 Nov 2025. <https://specs.ipfs.tech/ipips/ipip-0476/>.
- [21] IPFS Mainnet AutoConf. *Canonical AutoConf manifest (autoconf.json)*. Accessed 2026-02-28. <https://conf.ipfs-mainnet.org/autoconf.json>.
- [22] IPFS Kubo. *Kubo configuration: AutoConf*. Documentation (raw), accessed 2026-03-05. <https://raw.githubusercontent.com/ipfs/kubo/master/docs/config.md>.
- [23] IPFS Docs. *Kubo RPC API v0 reference: `expand-auto` resolves `auto` placeholders from AutoConf*. Updated 2025; accessed 2026-02-28. <https://docs.ipfs.tech/reference/kubo/rpc/>.
- [24] IPFS Kubo. *Kubo environment variables: `IPFS_HTTP_ROUTERS`, `IPFS_HTTP_ROUTERS_FILTER_PROTOCOLS`, and routing overrides*. Documentation (raw), accessed 2026-03-05. <https://raw.githubusercontent.com/ipfs/kubo/master/docs/environment-variables.md>.
- [25] IPFS Kubo. *Kubo v0.37 changelog: AutoConf control and explicit `["auto"]` placeholders*. 27 Aug 2025. <https://raw.githubusercontent.com/ipfs/kubo/master/docs/changelogs/v0.37.md>.
- [26] IPFS Rainbow. *CHANGELOG: AutoConf default `auto` expansion and control-plane default changes*. 2025–2026; accessed 2026-02-28. <https://github.com/ipfs/rainbow/blob/main/CHANGELOG.md>.
- [27] IPFS Mainnet AutoConf. *Registry and format notes (alpha; subject to change)*. 2025–2026; accessed 2026-02-28. <https://conf.ipfs-mainnet.org/>.
- [28] T. Pauly, M. Thomson, and C. A. Wood. Oblivious HTTP. RFC 9458, IETF, 2023.
- [29] M. Thomson and C. A. Wood. Discovering Oblivious Services via HTTP. RFC 9540, IETF, 2024.
- [30] I. Schasny and contributors. *Draft IPIP-0421: HTTP Delegated Routing Reader Privacy Upgrade*. Pull request #421 in *ipfs/specs*, June 2023 (draft / changes requested). <https://github.com/ipfs/specs/pull/421>.
- [31] IPFS Helia. *@helia/delegated-routing-v1-http-api-client documentation (browser caching defaults)*. API docs, accessed 2026-03-04. https://ipfs.github.io/helia-delegated-routing-v1-http-api/modules/_helia_delegated-routing-v1-http-api-client.html.

- [32] D. Schinazi, C. Wood, and J. Thomson (editors). *Chunked Oblivious HTTP Messages* (Internet-Draft). IETF, draft-ietf-ohai-chunked-ohhttp (latest), 2025–2026. <https://datatracker.ietf.org/doc/draft-ietf-ohai-chunked-ohhttp/>.
- [33] IPFS Kubo. *Release v0.40.1*. GitHub releases page (patch line), 27 Feb 2026. <https://github.com/ipfs/kubo/releases/tag/v0.40.1>.
- [34] IPFS Kubo. *Release v0.40.0: gateway exposes Routing V1 at /routing/v1 by default*. 25 Feb 2026. <https://github.com/ipfs/kubo/releases/tag/v0.40.0>.
- [35] IPFS Kubo. *Kubo configuration: Gateway.ExposeRoutingAPI*. Documentation (raw), accessed 2026-02-28. <https://raw.githubusercontent.com/ipfs/kubo/master/docs/config.md>.
- [36] IPFS Kubo. Kubo configuration reference (Routing section: path-based delegated routing, AutoConf structure, and accelerated DHT client caveats). <https://github.com/ipfs/kubo/blob/master/docs/config.md> (accessed 2026-03-05).
- [37] IPFS Rainbow. *Rainbow environment variables: delegated routing, RAINBOW_HTTP_ROUTERS, and autoconf*. Documentation, accessed 2026-03-05. <https://github.com/ipfs/rainbow/blob/main/docs/environment-variables.md>.
- [38] IPFS Kubo. Kubo configuration reference (HTTPRetrieval.Enabled). <https://github.com/ipfs/kubo/blob/master/docs/config.md> (accessed 2026-03-05).
- [39] IPFS Standards. *Trustless Gateway Specification*. <https://specs.ipfs.tech/http-gateways/trustless-gateway/> (accessed 2026-03-05).